

CS675

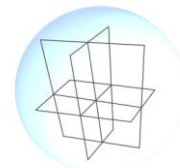
Computer Graphics

Hierarchical Modelling

Parag Chaudhuri



Department of Computer Science and Engineering
IIT Bombay



ViGIL
VISION, GRAPHICS,
IMAGING & LEARNING

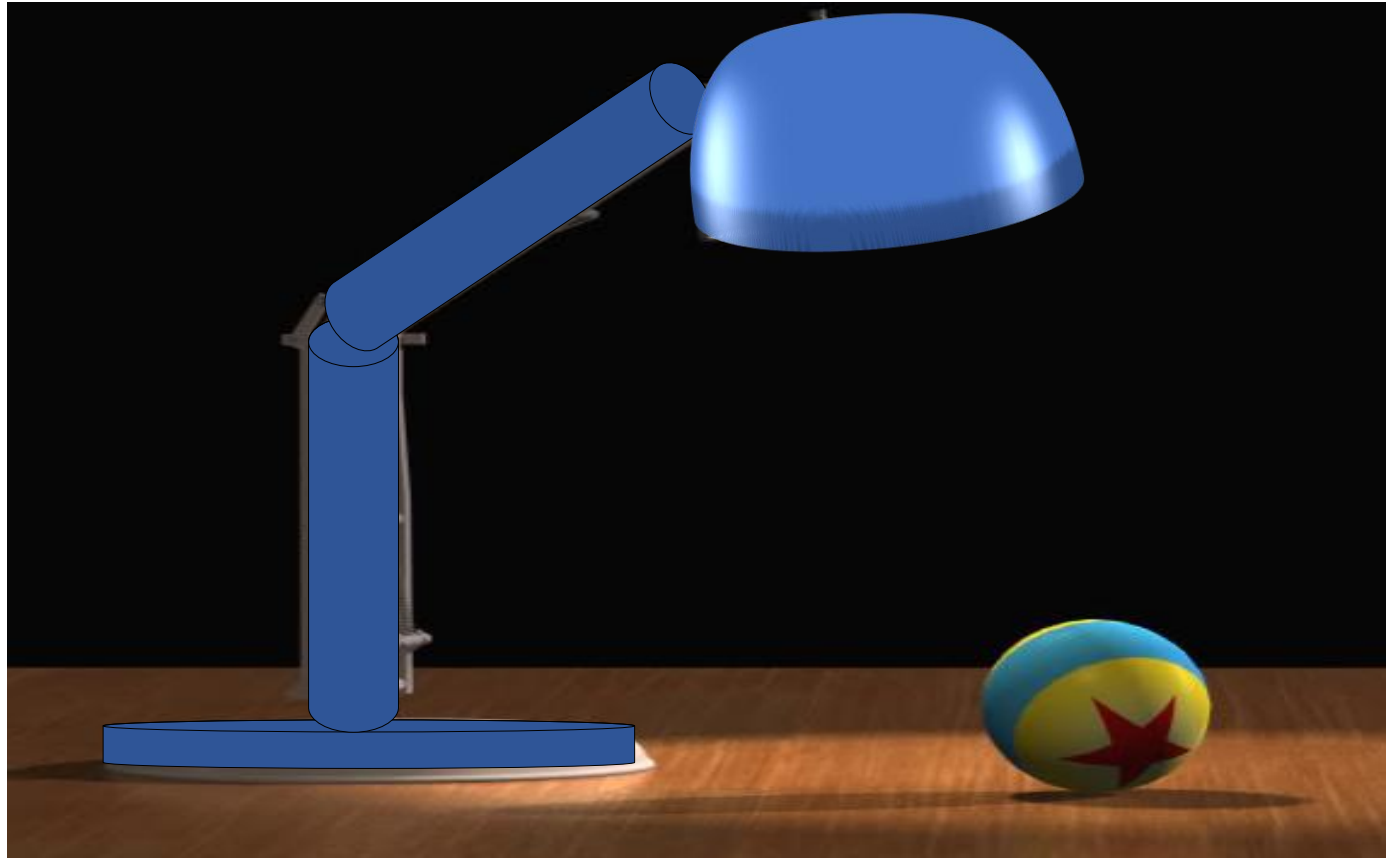
Modelling

- Modelling and Rendering
- Transformations



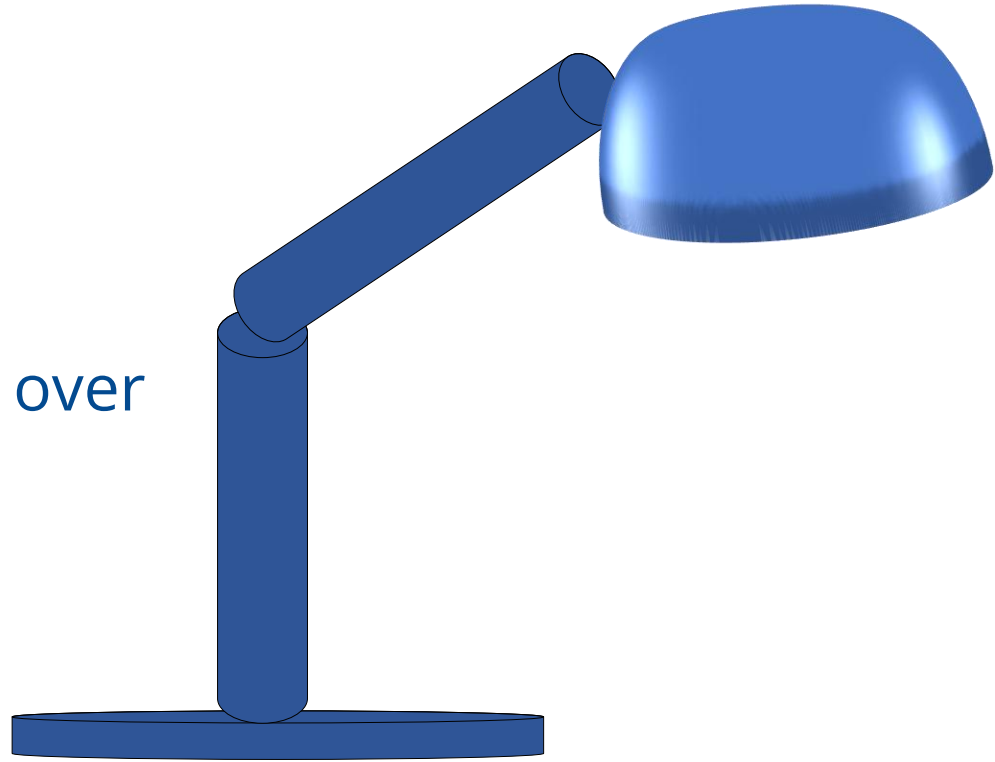
Modelling

- Modelling and Rendering
- Transformations



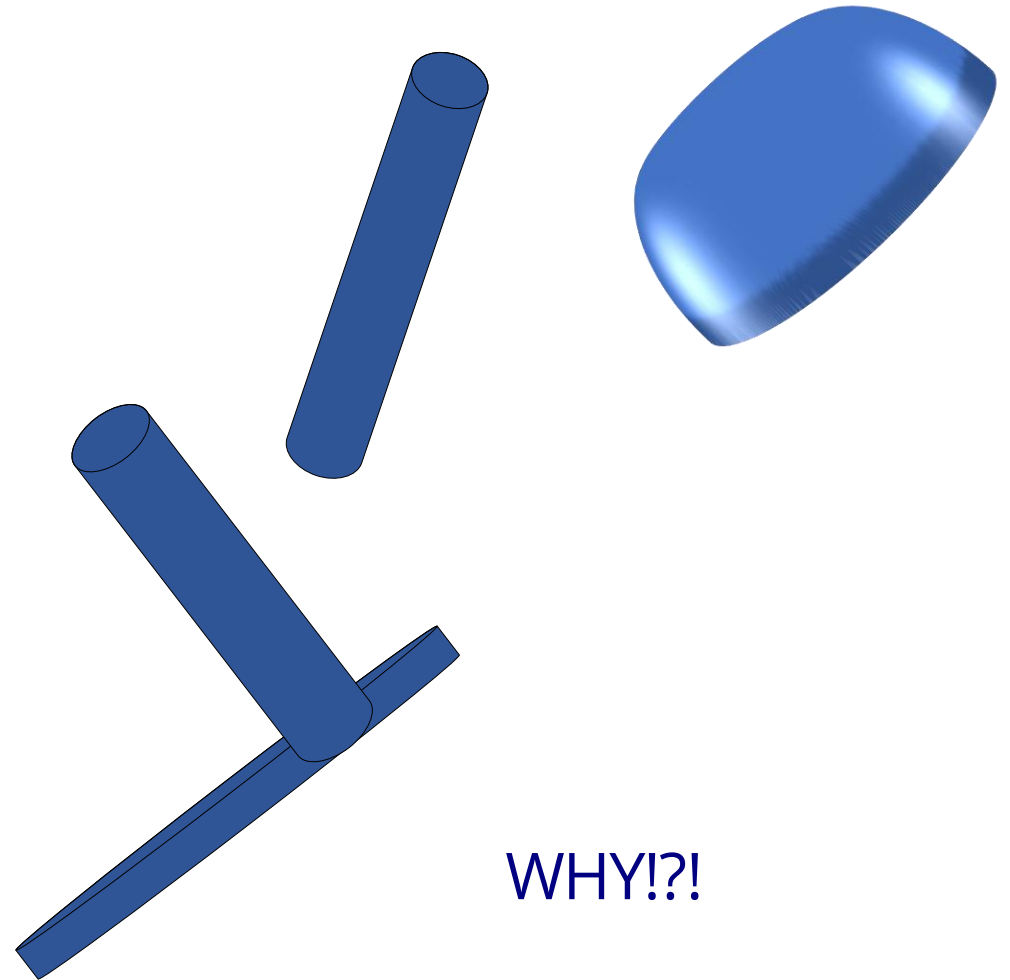
Modelling

- Modelling and Rendering
- Transformations
- Moving this model?
 - Change the transformations over time.



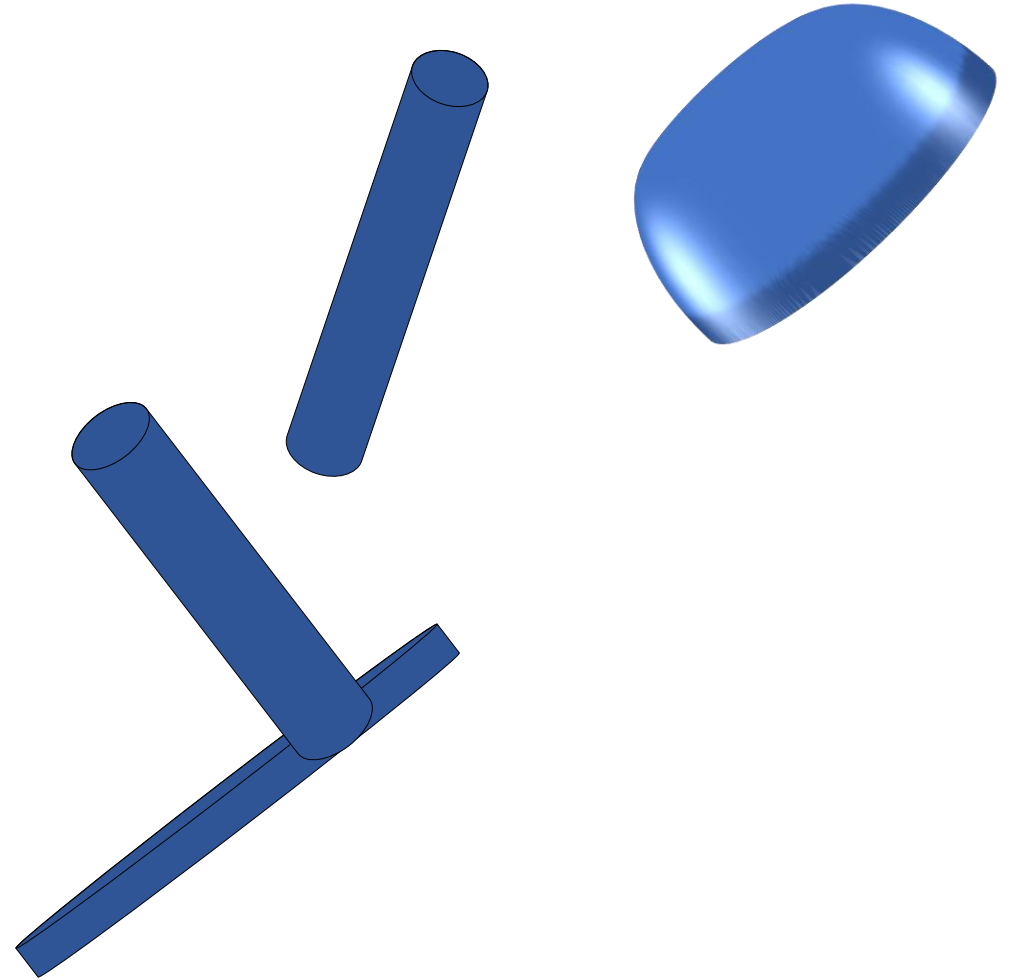
Modelling

- Modelling and Rendering
- Transformations
- Moving this model?
 - Change the transformations over time.
- Model falls apart!



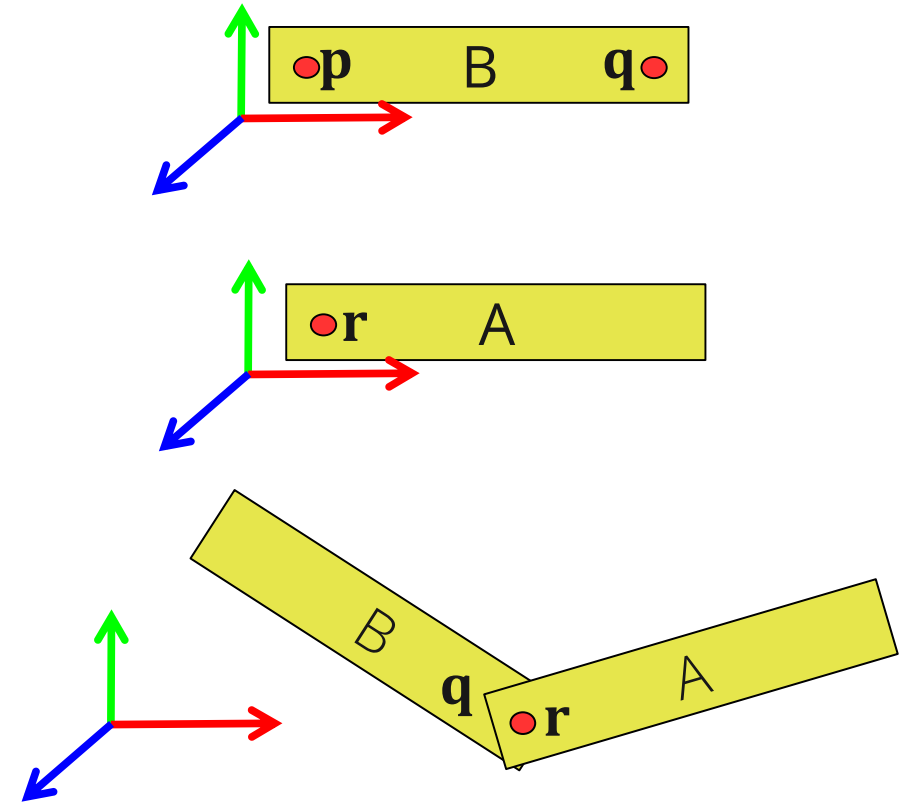
Modelling

- The object we are modelling is *constrained* but the model does not know that.
- We need:
 - To represent the structure of the model.
 - A handle on parameters so that we can move only through valid poses.
- So we structure our transformations into a hierarchy.



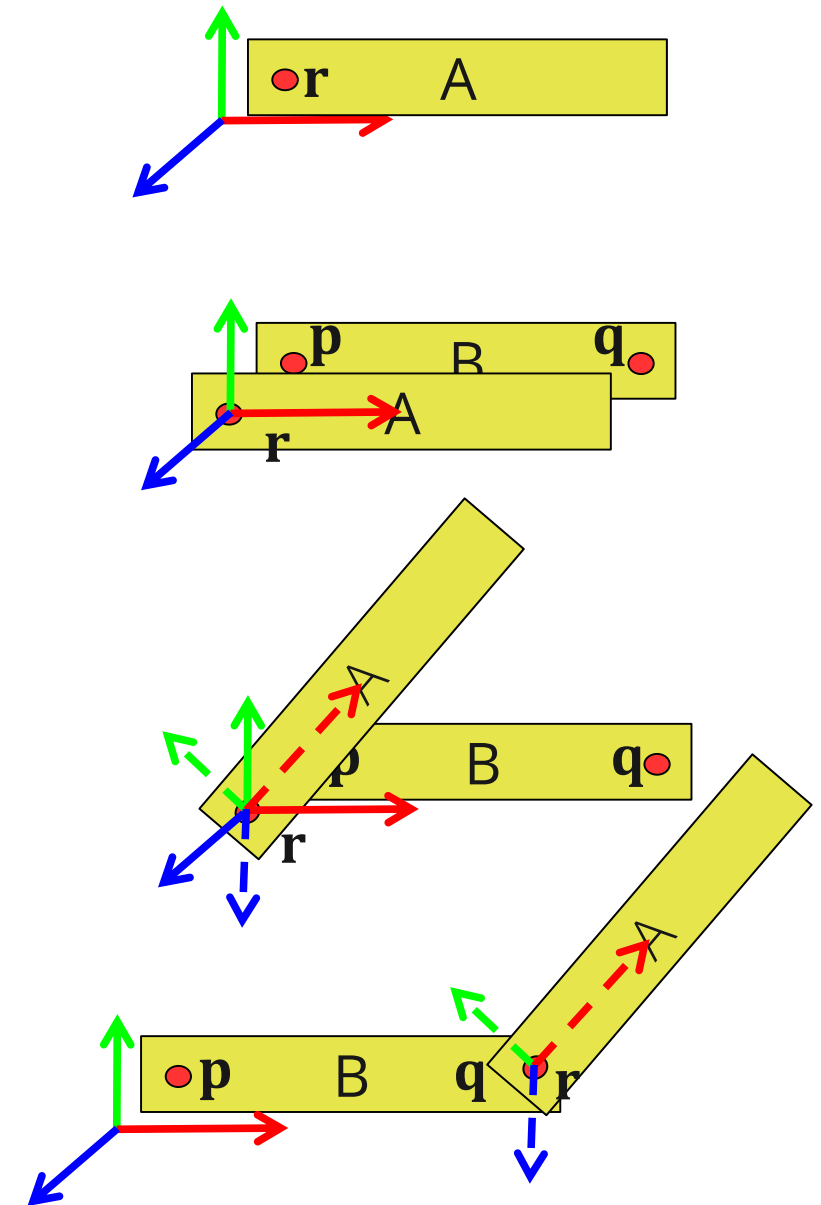
Hierarchical Modelling

- Modelling a two-link arm
 - Rigid Links
 - Hinge Joints (1 degree of freedom or *dof*)
- Upper arm link B has two joints p and q (shoulder and elbow)
- Lower arm link A has one joint, r
- Attach point q on B to r on A.
- Parameters to control –
 - shoulder position T
 - shoulder angle θ (A and B together rotate about p)
 - elbow angle φ (A rotates about r, and stays attached to B at q)



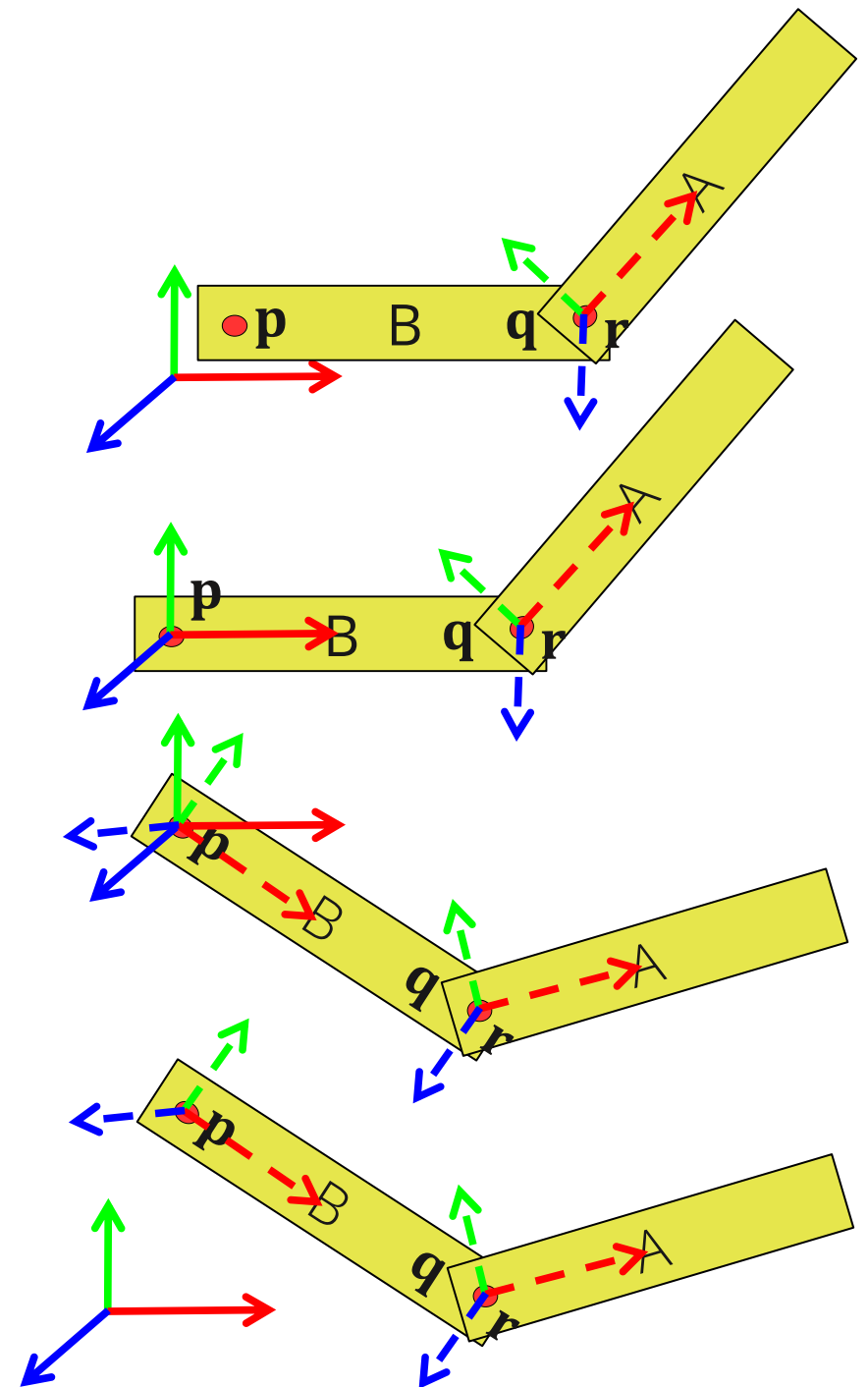
Hierarchical Modelling

- Modelling a two-link arm
 - Start with A and B in their original positions
 - Apply only to A
 - Translate by $-r$
 - Rotate by φ about the origin.
 - Translate by q , bringing r and q together.
 - We can now consider q as the origin of the lower arm link, and regard A as being in this coordinate system.



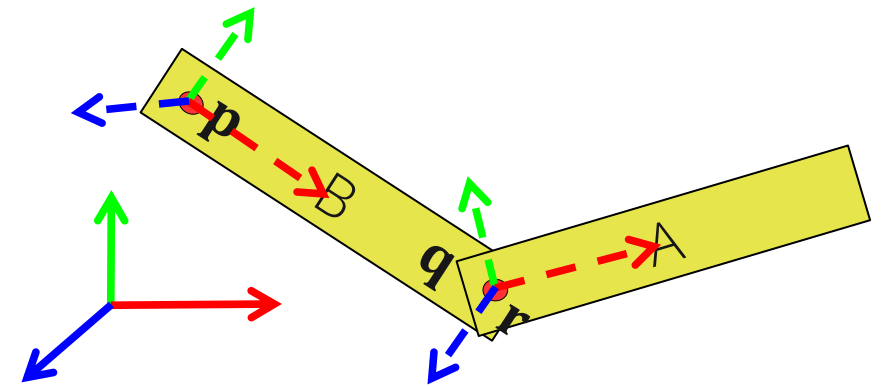
Hierarchical Modelling

- Modelling a two-link arm
 - Now the transformations apply to both A and B
 - Translate by $-p$
 - Rotate by θ about the origin.
 - Translate by T to place the two link arm at the proper position.



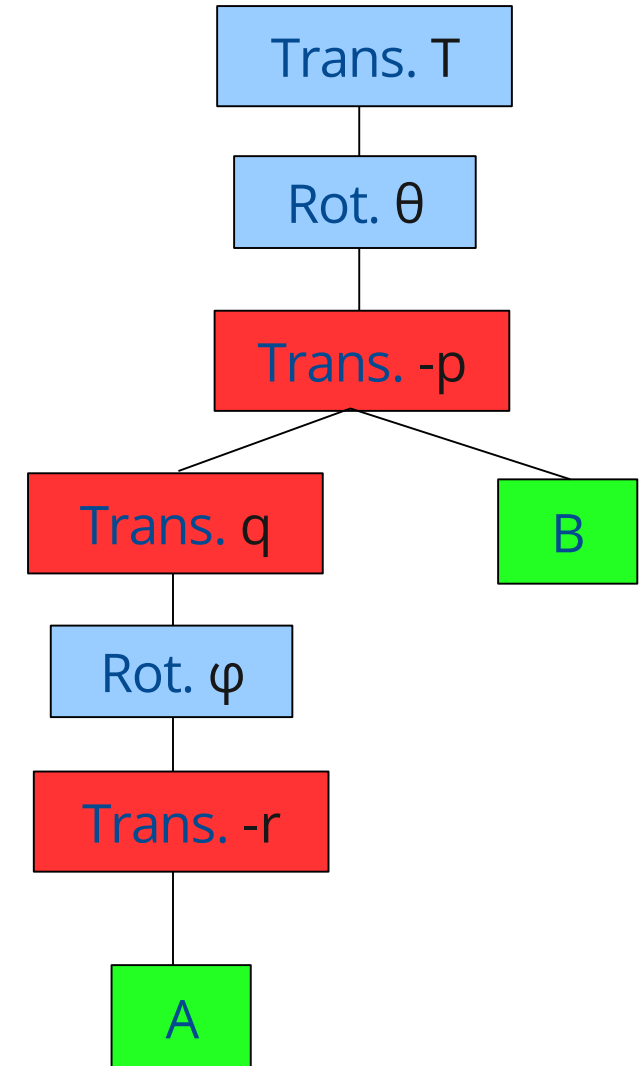
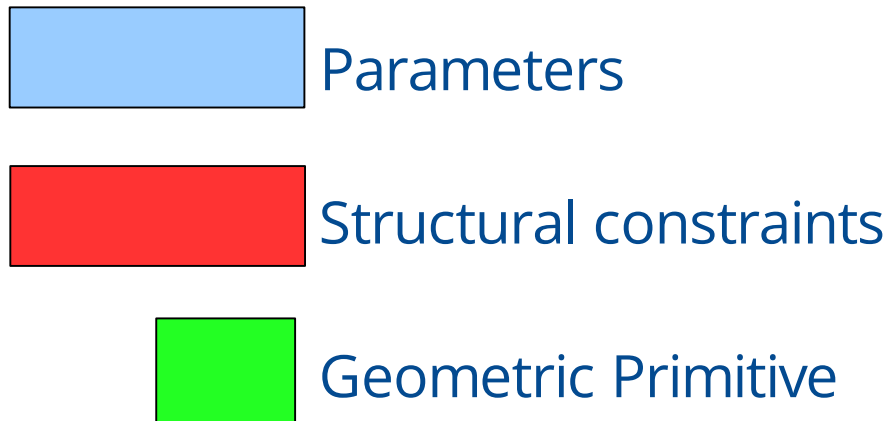
Hierarchical Modelling

- Modelling a two-link arm
 - Complicated?
 - Remember the sequence of transformations and parameters
 - Re-apply all transformations in same sequence when parameters change
- Note:
 - θ , φ and T are parameters – we change these to animate the model
 - p , q and r are structural constraints. If we change them – model falls apart.



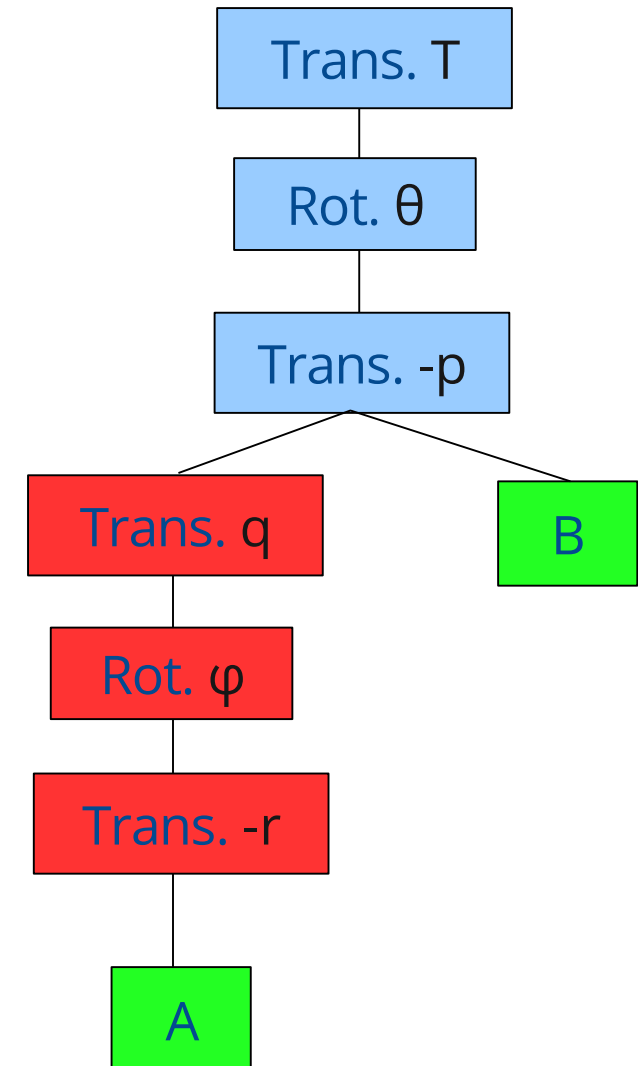
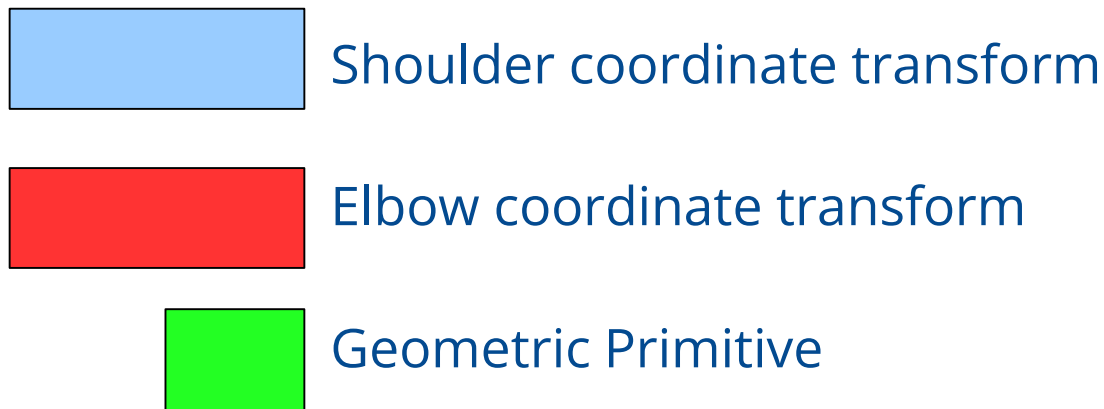
Hierarchical Modelling

- Store the modelling sequence in a hierarchy
- Leaves have the geometry.
- Internal nodes have transformations.
- Transformations apply to everything under them – start at the bottom and work your way up.



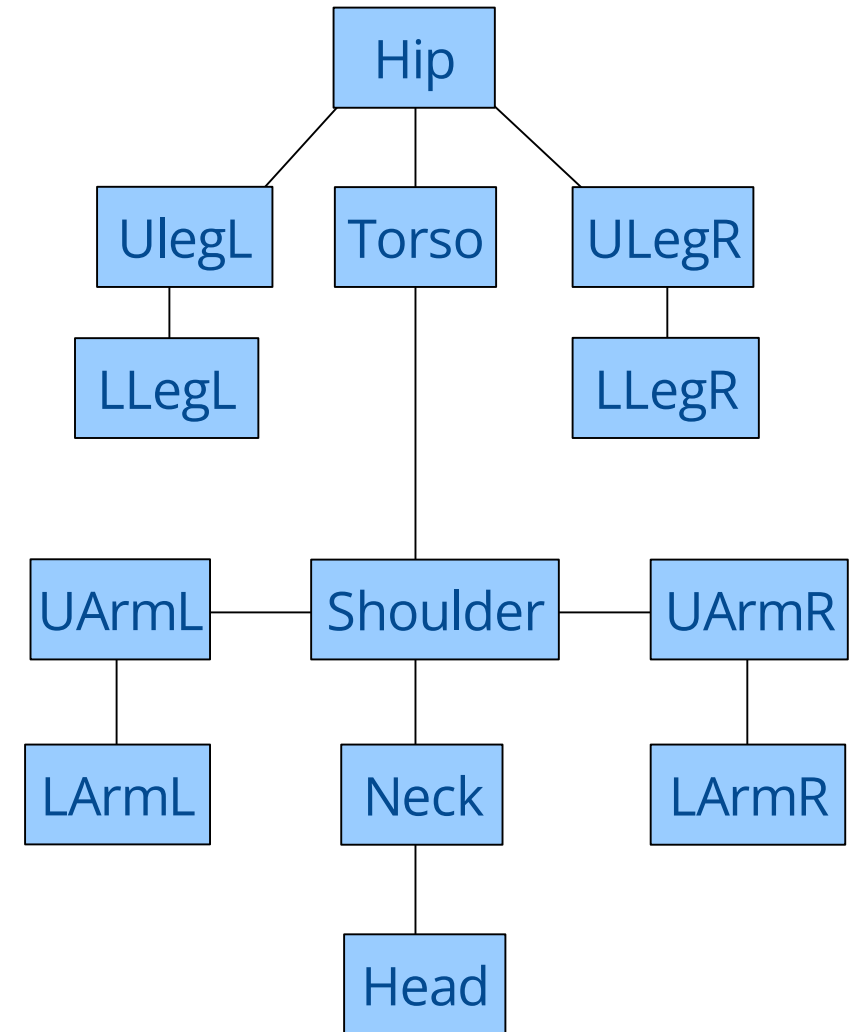
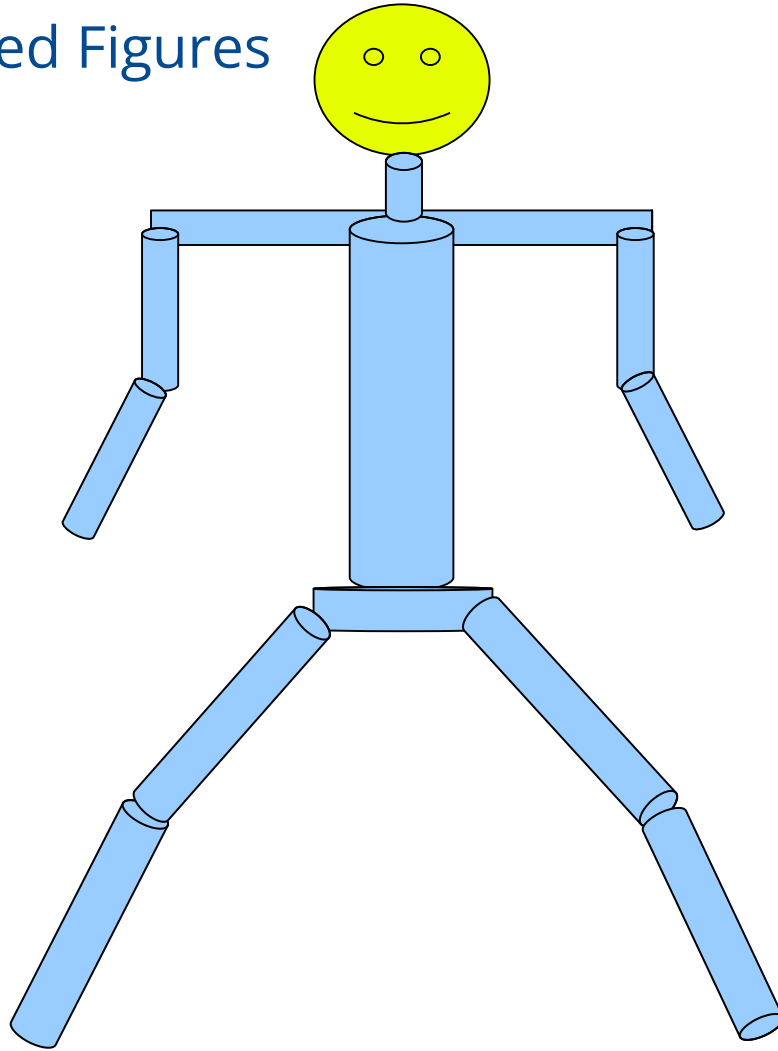
Hierarchical Modelling

- Another view
 - The shoulder coordinate transformation moves everything below it w.r.t. the shoulder:
 - B
 - A and its transformation
 - The elbow coordinate transform moves A with respect to the shoulder coordinate transform.



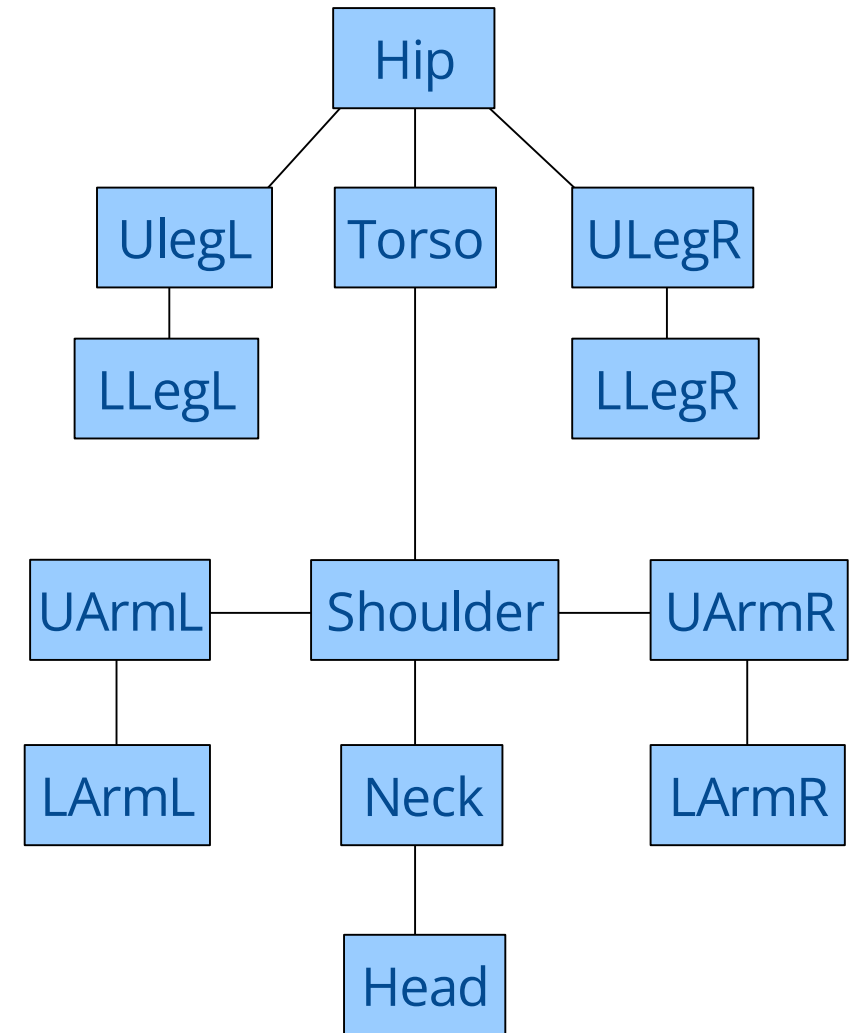
Hierarchical Modelling

- Articulated Figures



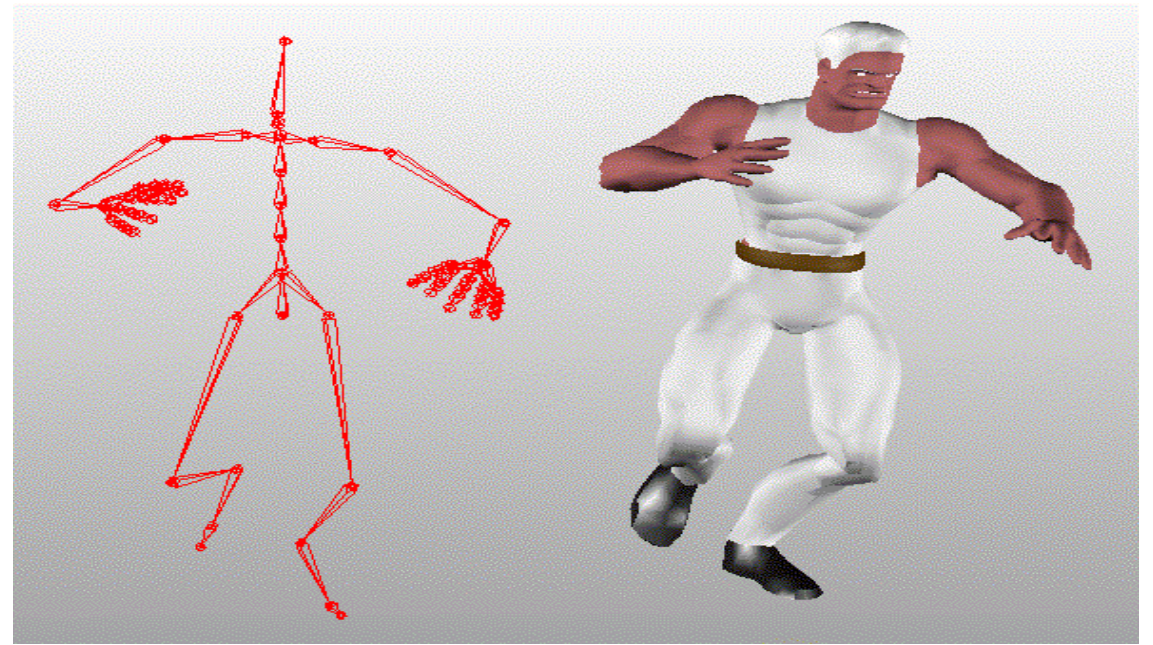
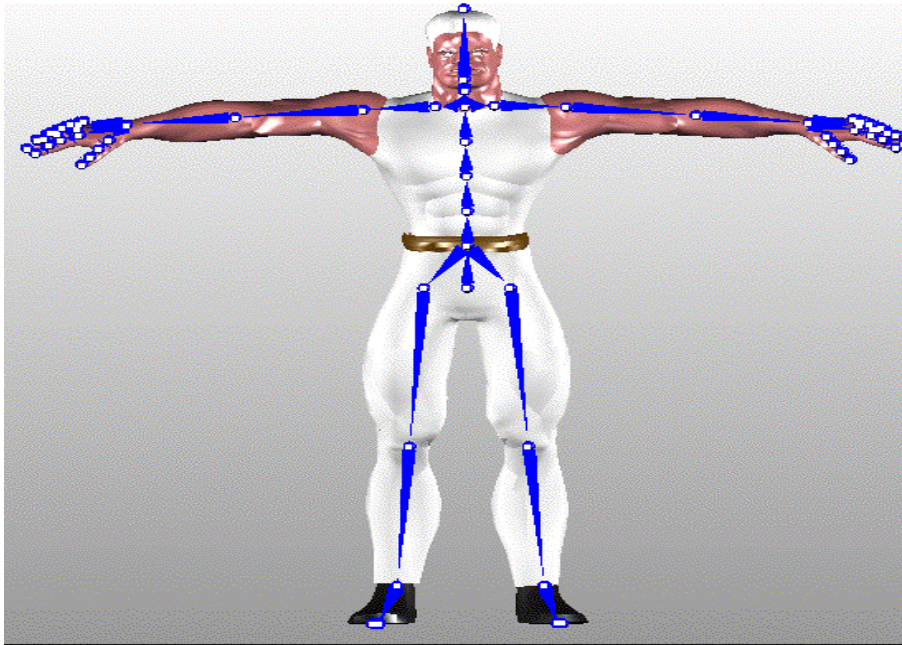
Hierarchical Modelling

- Articulated Figures
 - Each node represents the geometry, rotation parameters and structural transformations.
 - Root can be anywhere – here it is at the hip.
 - A realistic human is much more complex
 - Difficult to control so many DoF's (later problem)
 - A Directed Acyclic Graph
 - Not necessarily a tree (why not?)



Hierarchical Modelling

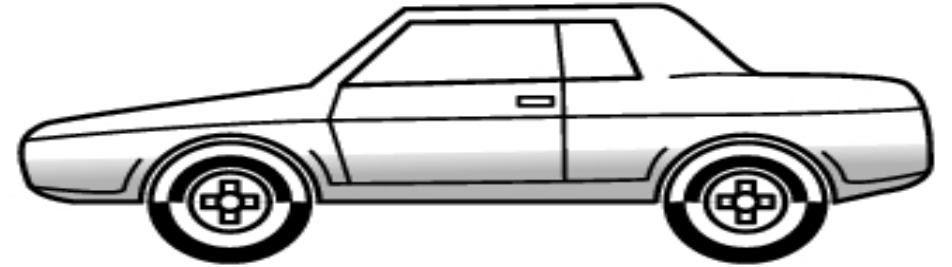
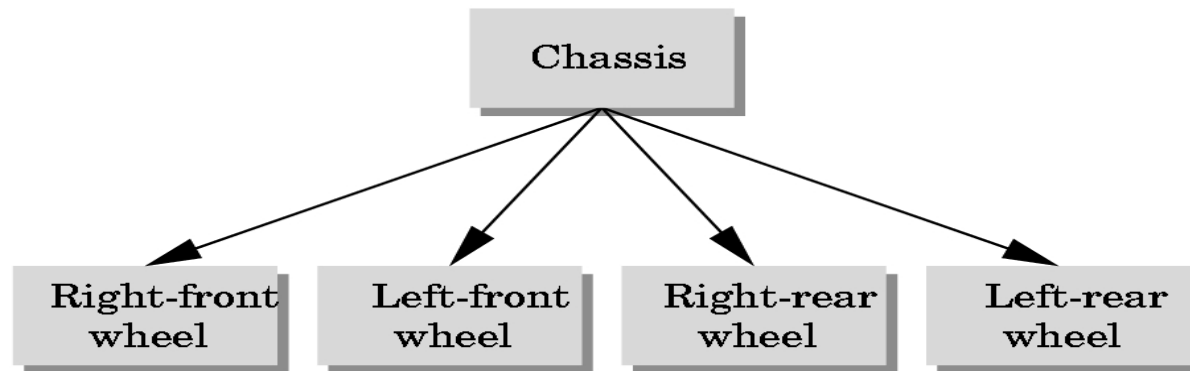
- Articulated Figures
- Character Rigging and skinning



<http://www.okino.com/conv/skinning.htm>

Hierarchical Modelling

- We can model a lot of things this way



Hierarchical Modelling

We can model a lot of things this way



LUXO Jr., PIXAR Animation Studios



<https://www.gt-rider.com/se-asia-motorcycling/threads/fmx-freestyle-motocross-in-my-home.5746/>



Wall-E, PIXAR Animation Studios, 2008



<http://thechainring.com/complete-bicycles-frames-and-forks/complete-unicycles/nimbus-red-24-unicycle/>

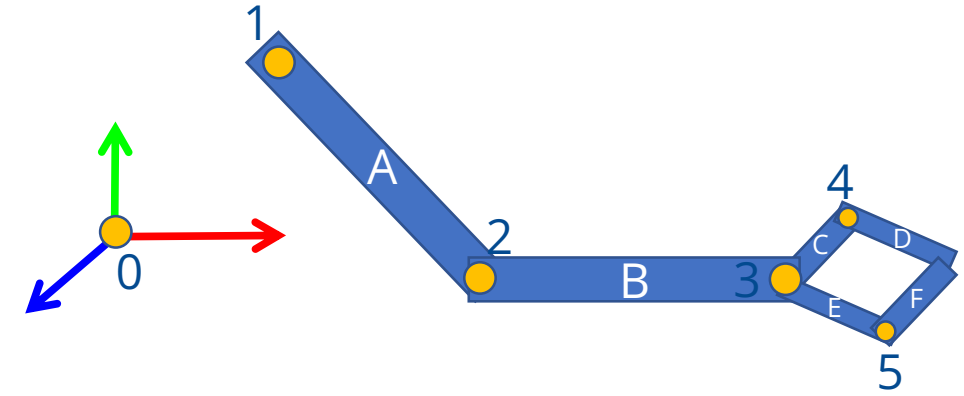


<https://www.lego.com/en-us/themes/minifigures>

Implementing Hierarchical Modeling

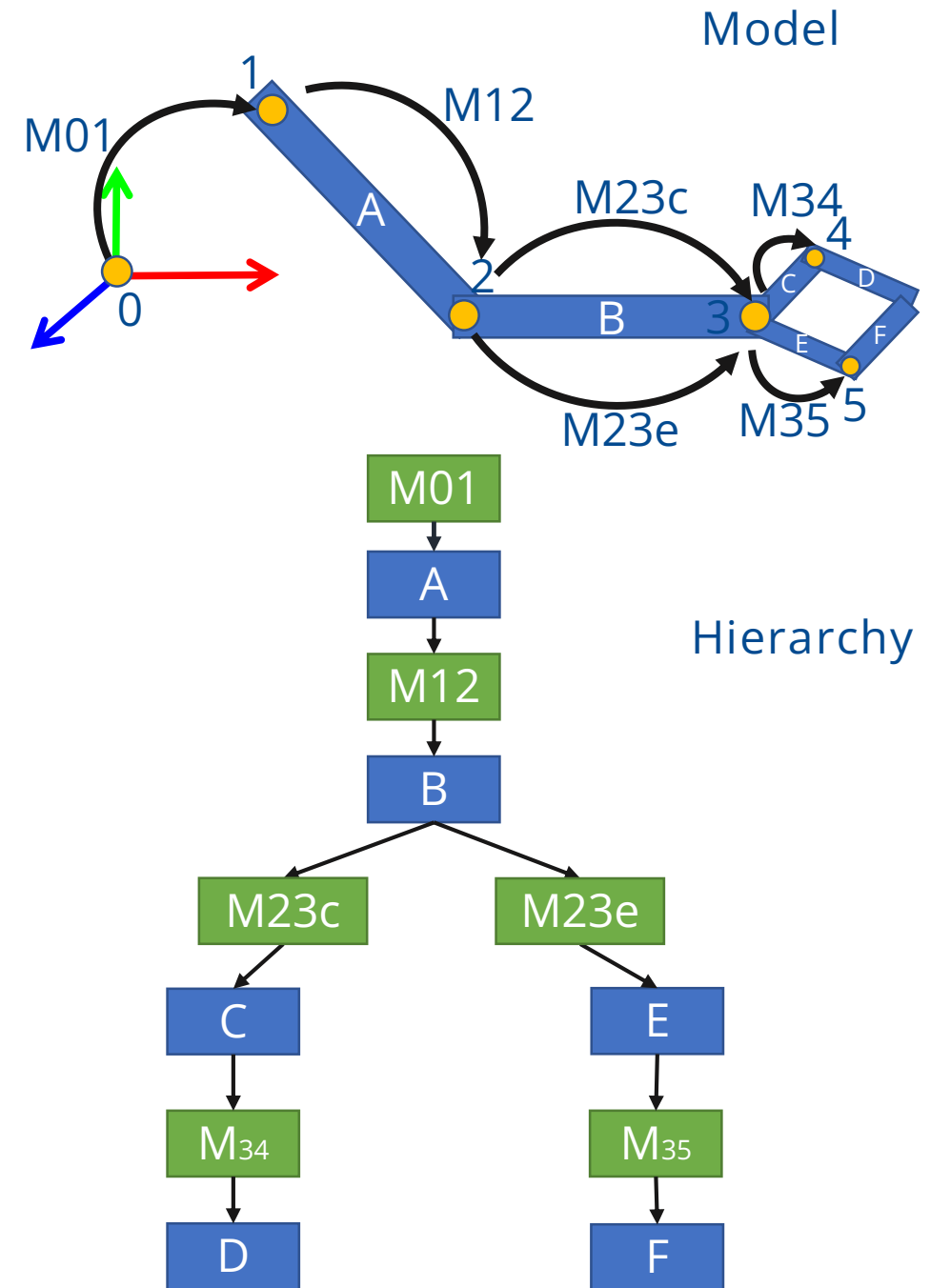
Hierarchical Modelling

- Model of an arm
 - Rigid Links A, B, C, D, E, F
 - Hinge Joints 1, 2, 3, 4, 5



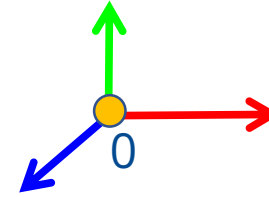
Hierarchical Modelling

- Model of an arm
 - Rigid Links A, B, C, D, E, F
 - Hinge Joints 1, 2, 3, 4, 5
- Matrix transformation from world origin, 0 to root joint 1 is M_{01} .
- Matrix transformation from joint 1 to 2 is M_{12} and so on.
- Assume current transformation matrix is T
- Whatever we draw, it draws under T .



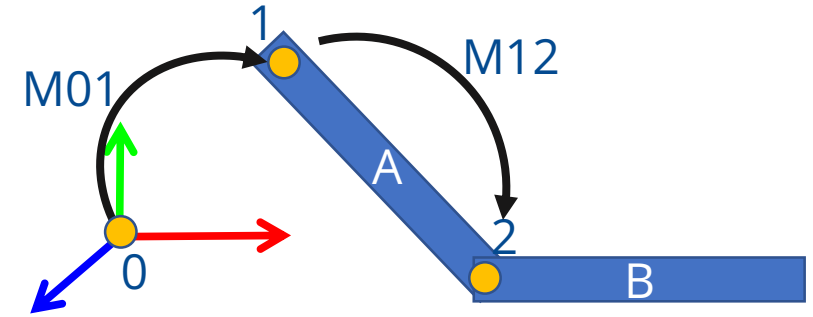
Hierarchical Modelling

- We start at 0, $T = I$
- PushMatrix() //PushMatrix saves T to the stack
 - $T = T.M01$
 - Draw A
 - $T = T.M12$
 - Draw B
 - PushMatrix()
 - $T = T.M23c$
 - Draw C
 - PushMatrix()
 - $T = T.M34$
 - Draw D
 - PopMatrix() //PopMatrix overwrites T with the popped matrix
 - PopMatrix()
 - PushMatrix()
 - $T = T.M23e$
 - Draw E
 - PushMatrix()
 - $T = T.M35$
 - Draw F
 - PopMatrix()
 - PopMatrix
- PopMatrix()



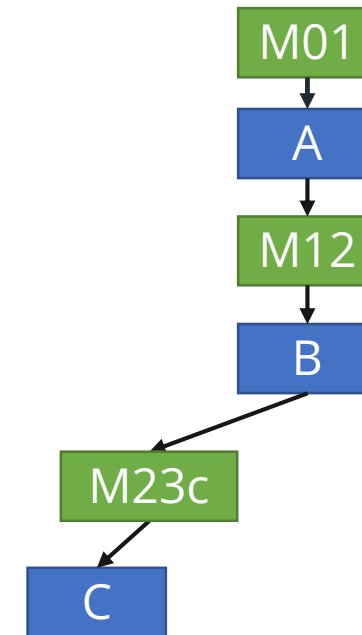
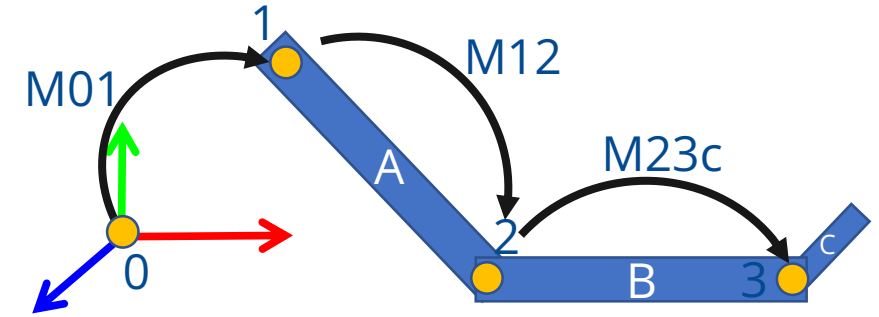
Hierarchical Modelling

- We start at 0, $T = I$
- PushMatrix() //PushMatrix saves T to the stack
 - $T = T.M01$
 - Draw A
 - $T = T.M12$
 - Draw B
- PopMatrix() //PopMatrix overwrites T with the popped matrix
- PushMatrix()
 - $T = T.M23c$
 - Draw C
 - PushMatrix()
 - $T = T.M34$
 - Draw D
 - PopMatrix() //PopMatrix overwrites T with the popped matrix
- PopMatrix()
- PushMatrix()
 - $T = T.M23e$
 - Draw E
 - PushMatrix()
 - $T = T.M35$
 - Draw F
 - PopMatrix()
- PopMatrix()
- PopMatrix()



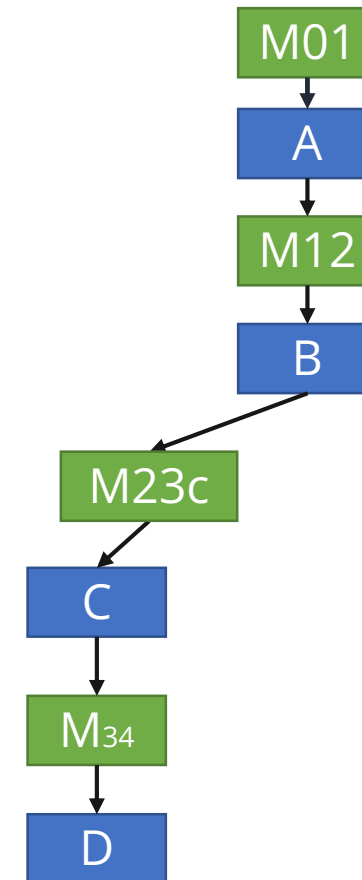
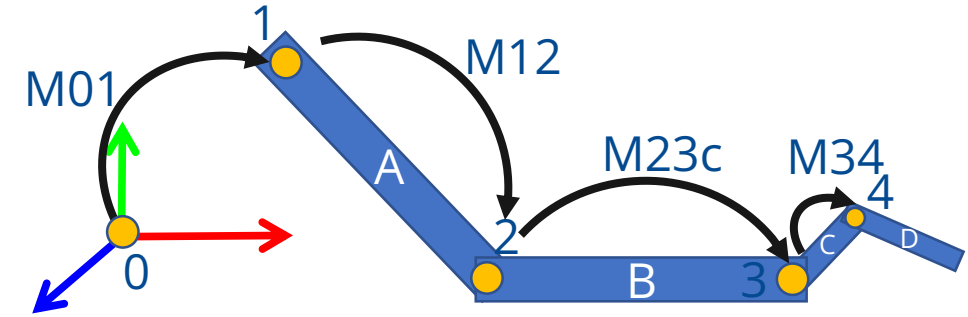
Hierarchical Modelling

- We start at 0, $T = I$
- PushMatrix() //PushMatrix saves T to the stack
 - $T = T.M01$
 - Draw A
 - $T = T.M12$
 - Draw B
 - PushMatrix()
 - $T = T.M23c$
 - Draw C
 - PushMatrix()
 - $T = T.M34$
 - Draw D
 - PopMatrix() //PopMatrix overwrites T with the popped matrix
 - PopMatrix()
 - PushMatrix()
 - $T = T.M23e$
 - Draw E
 - PushMatrix()
 - $T = T.M35$
 - Draw F
 - PopMatrix()
 - PopMatrix
 - PopMatrix()



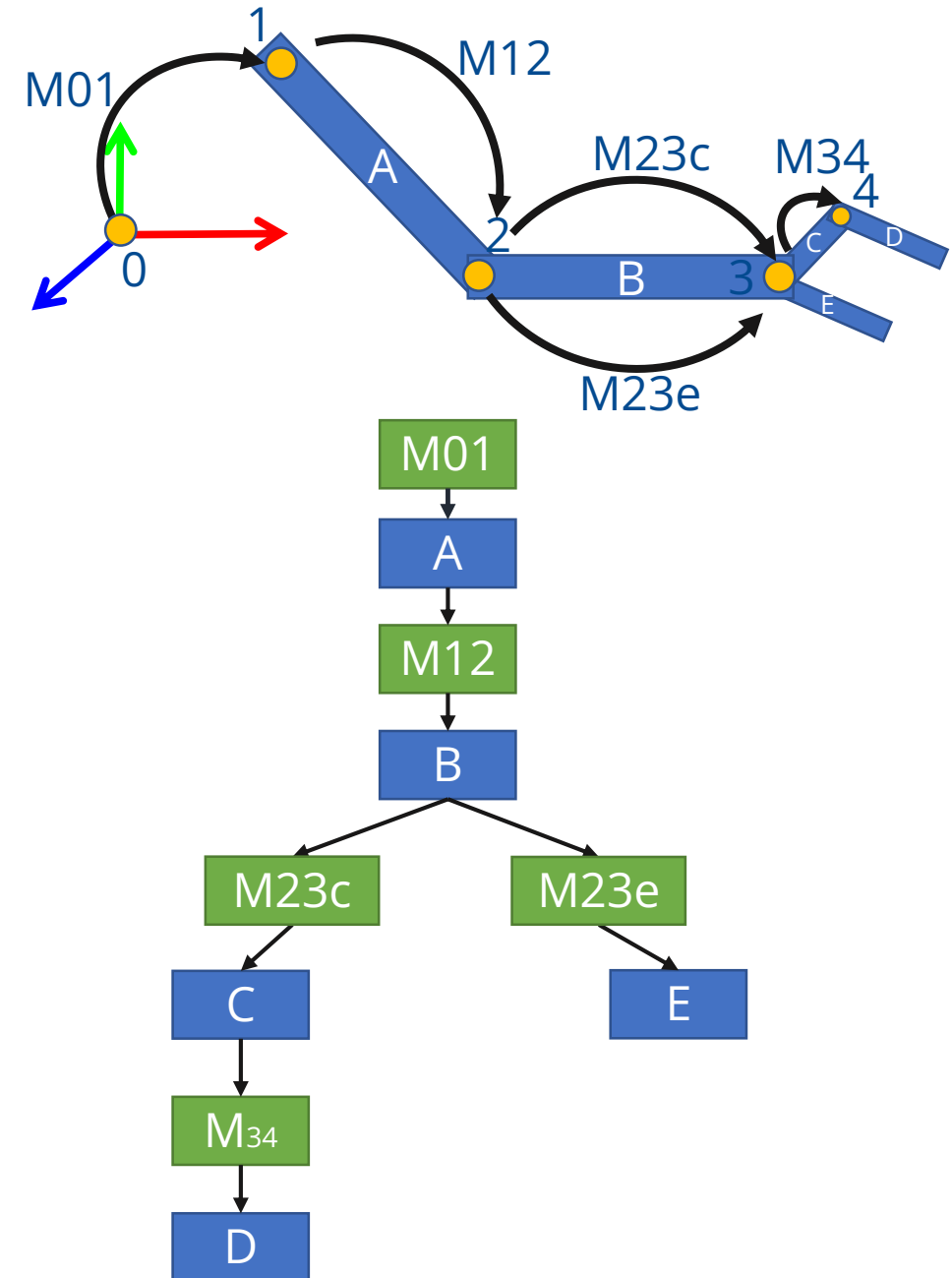
Hierarchical Modelling

- We start at 0, $T = I$
- PushMatrix() //PushMatrix saves T to the stack
 - $T = T.M01$
 - Draw A
 - $T = T.M12$
 - Draw B
 - PushMatrix()
 - $T = T.M23c$
 - Draw C
 - PushMatrix()
 - $T = T.M34$
 - Draw D
 - PopMatrix() //PopMatrix overwrites T with the popped matrix
 - PopMatrix()
 - PushMatrix()
 - $T = T.M23e$
 - Draw E
 - PushMatrix()
 - $T = T.M35$
 - Draw F
 - PopMatrix()
 - PopMatrix
 - PopMatrix()



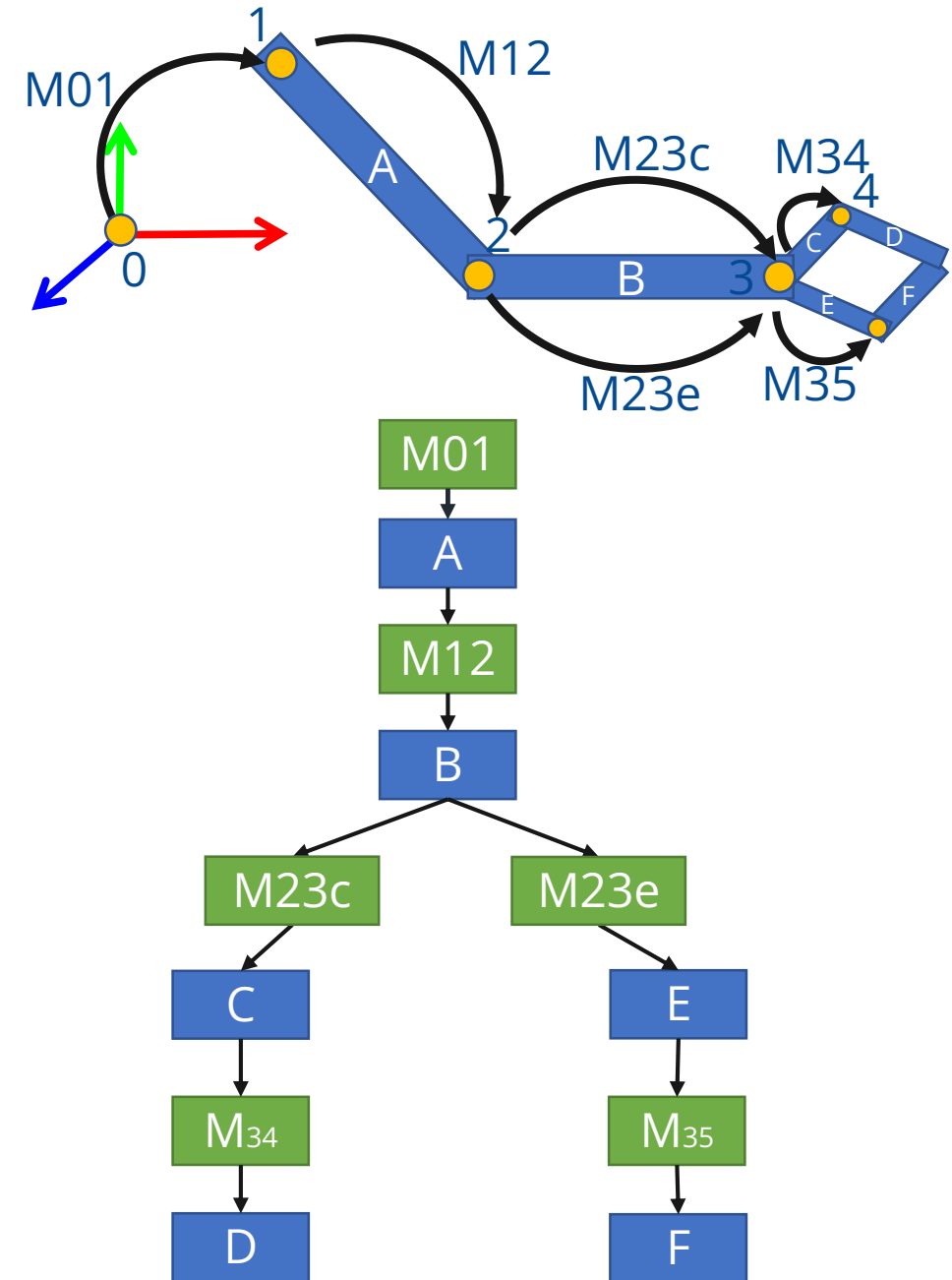
Hierarchical Modelling

- We start at 0, $T = I$
- PushMatrix() //PushMatrix saves T to the stack
 - $T = T.M01$
 - Draw A
 - $T = T.M12$
 - Draw B
 - PushMatrix()
 - $T = T.M23c$
 - Draw C
 - PushMatrix()
 - $T = T.M34$
 - Draw D
 - PopMatrix() //PopMatrix overwrites T with the popped matrix
 - PopMatrix()
 - PushMatrix()
 - $T = T.M23e$
 - Draw E
 - PushMatrix()
 - $T = T.M35$
 - Draw F
 - PopMatrix()
 - PopMatrix
 - PopMatrix()



Hierarchical Modelling

- We start at 0, $T = I$
- PushMatrix() //PushMatrix saves T to the stack
 - $T = T.M01$
 - Draw A
 - $T = T.M12$
 - Draw B
 - PushMatrix()
 - $T = T.M23c$
 - Draw C
 - PushMatrix()
 - $T = T.M34$
 - Draw D
 - PopMatrix() //PopMatrix overwrites T with the popped matrix
 - PopMatrix()
 - PushMatrix()
 - $T = T.M23e$
 - Draw E
 - PushMatrix()
 - $T = T.M35$
 - Draw F
 - PopMatrix()
 - PopMatrix
- PopMatrix()

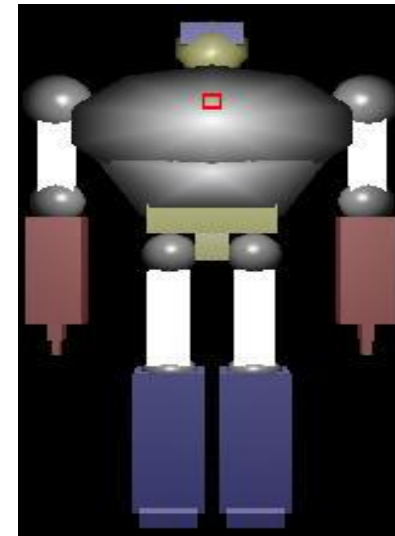
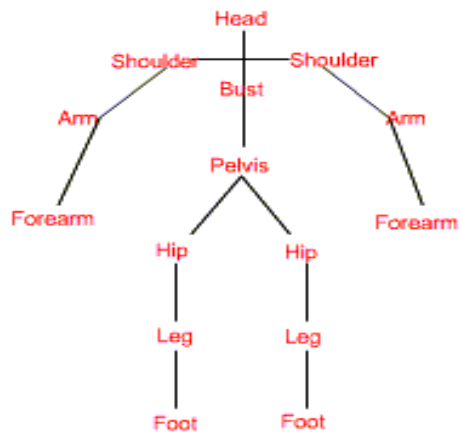


Hierarchical Modelling

- Doing this in OpenGL 2.x and earlier
 - Use the internal Matrix Stack.
 - Current matrix is automatically product of everything already on the stack.
 - This is the matrix on top of the stack.
- For OpenGL 3+ implement this yourself.
- Algorithm
 - Load Identity Matrix
 - For each internal node
 - Push new matrix into stack
 - Concatenate transformations onto current matrix.
 - Recursively descend tree
 - Pop matrix off stack
 - For each leaf node
 - Draw the geometry using the current transformation

Hierarchical Modelling

- Doing this in OpenGL
- Using VAO, VBO and shaders
- See Tutorial 7: https://github.com/paragchaudhuri/cs475-tutorials/tree/master/Tutorial_07



<http://www.gamedev.net/reference/articles/article1267.asp>

This example uses deprecated OpenGL but the hierarchical model organization is still valid.